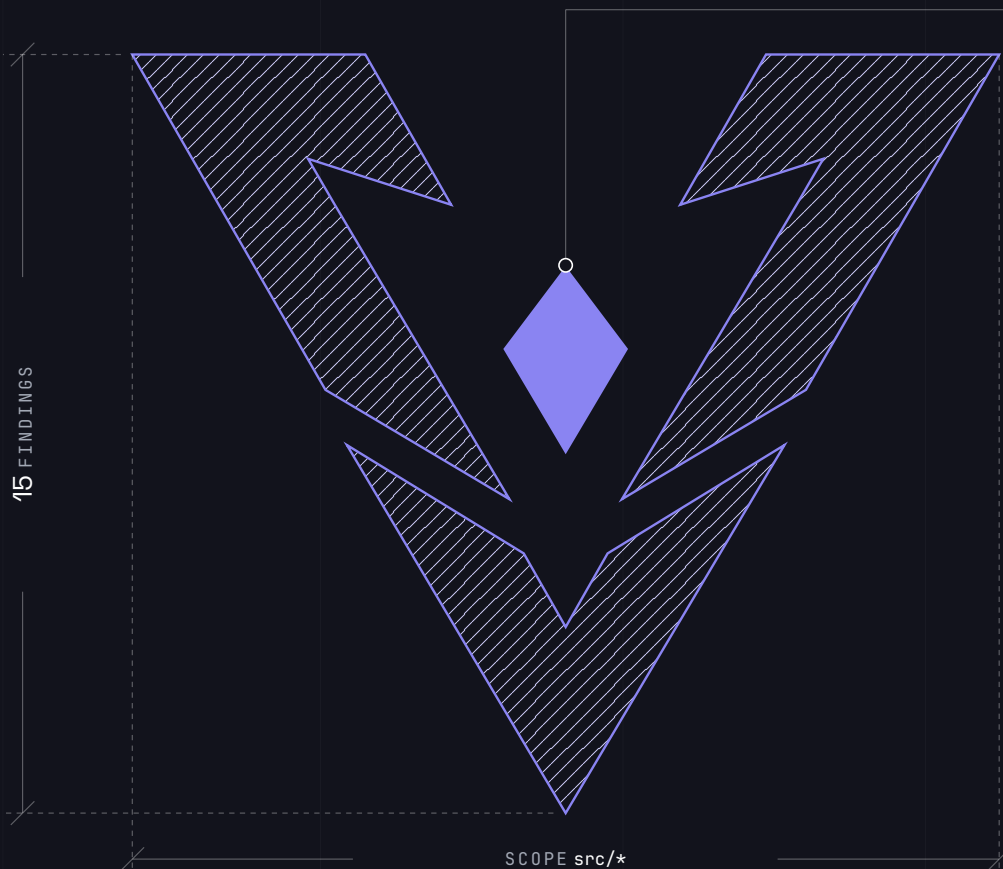


COMMIT E3B5D2C



SECURITY REVIEW

Need4Audit.

March 23rd 2026 - March 29th 2026

Contents.

01	About Valves Security	3
02	Disclaimer	3
03	Risk Classification	3
04	Protocol Summary	4
05	Executive Summary	4
06	Findings	5
	■ High Findings	5
	[H-01] Rejected reports don't block refund, letting companies kill researcher escalation rights	5
	[H-02] Program deactivation after payout blocks remaining researchers from escalating	6
	■ Medium Findings	7
	[M-01] No bounty reservation in <code>finalizeEscalation</code> creates a race-to-drain	7
	[M-02] Second opinion requests are incorrectly blocked when no alternate payout tier exists	7
	[M-03] Downgrade fallback silently rewrites judge's severity choice	8
	[M-04] Shared judge pool without per-report reservation blocks settlement	9
	[M-05] Penalty-only deposit reverts, blocking program reactivation	10
	[M-06] Judge fee design biases outcomes toward approval	11
	[M-07] Active programs can't be topped up with additional deposits	12
	[M-08] Centralization risk	12
	■ Low/Info Findings	13
	[L-01] Fee-on-transfer tokens would break escrow accounting	13
	[L-02] No <code>cancelRefund</code> function makes PAUSED state irreversible	14
	[L-03] No resolution deadline for reports after judge assignment	14
	[I-01] Never-pruned <code>programPenaltyReports</code> causes unbounded gas growth	15
	[I-02] Faucet <code>Withdrawn</code> event logs wrong amount	15

SECTION 01

About Valves Security.

Valves Security is an independent smart contract security firm founded by Veselin and Valeri. We work with security researchers specializing in Solidity, Rust and DAML. Every engagement is led by both co-founders with senior security researchers: we review the code line by line, substantiate every reported issue and remain involved through remediation.

To request a security review, visit valvessecurity.com or contact @ValvesSec on X.

SECTION 02

Disclaimer.

This report reflects a time-boxed review of the code at the commit listed in the Executive Summary, limited to the security of the files in scope. A security review cannot guarantee the absence of vulnerabilities, and this report is neither an endorsement of the project, its team or any associated token, nor investment advice. Changes made after the reviewed commit may introduce new issues; we recommend follow-up reviews and an ongoing bug bounty program.

SECTION 03

Risk Classification.

LIKELIHOOD / IMPACT	IMPACT: HIGH	IMPACT: MEDIUM	IMPACT: LOW
High	■ Critical	■ High	■ Medium
Medium	■ High	■ Medium	■ Low
Low	■ Medium	■ Low	■ Low

Impact

- **High** — leads to a significant material loss of assets in the protocol or a group of users.
- **Medium** — leads to a moderate material loss of assets in the protocol, moderately harms a group of users or causes some disruption of the protocol's functionality.
- **Low** — no funds at risk.

Likelihood

- **High** — the attack path is possible with reasonable assumptions. The issue is almost certain to happen.
- **Medium** — only a conditionally incentivized attack vector, but still relatively likely.
- **Low** — has unlikely assumptions or requires a significant stake by the attacker.

SECTION 04

Protocol Summary.

Need4Audit is a Web3 security audit marketplace that connects projects with vetted security researchers through a reputation-driven network. By aggregating verified credentials and enabling structured audit requests, it streamlines auditor discovery and matching without intermediaries, turning fragmented security talent into an efficient, on-demand layer of trusted expertise.

SECTION 05

Executive Summary.

OVERVIEW

PROJECT	REVIEW PERIOD
Need4Audit	March 23rd 2026 - March 29th 2026
COMMIT	
e3b5d2c3e2599d94afbbae9e8d0b0edbdd0f220d	

SCOPE

ID	FILES IN SCOPE
01	src/*

SECTION 06

Findings.

■ High Findings

[H-01] Rejected reports don't block refund, letting companies kill researcher escalation rights

HIGH

FIXED

Impact: High**Likelihood:** Medium**Description**

When a primary judge rejects a report (severity = 0), the report moves from `SUBMITTED` to `REJECTED`. The problem is that `REJECTED` isn't included in `_isBlockingStatus()`, so `blockingReportCount` drops to zero immediately on rejection.

Meanwhile, `escalateReport()` does allow escalation of rejected reports, but it also requires the program to be `ACTIVE`. This opens up a timing attack:

1. Researcher submits a report. `blockingReportCount = 1`.
2. Primary judge rejects it. Status becomes `REJECTED`, `blockingReportCount = 0`.
3. Researcher still has time to escalate (until `timeLockEnd`).
4. Company sees zero blocking reports and calls `initiateRefund()`. Program goes to `PAUSED`.
5. Researcher tries to escalate — reverts with `PROGRAM_NOT_ACTIVE`.
6. Escalation right is permanently lost.

The 5-day `REFUND_WINDOW` before `executeRefund` doesn't help here because `initiateRefund` alone is enough to flip the program to `PAUSED`, which already blocks escalation.

Recommendations

Allow `escalateReport()` to work when the program is `PAUSED` or `DRAFT`, not just `ACTIVE`:

```
1  require(  
2    -    config.status == BountyProgramRegistry.Status.ACTIVE,  
3    +    config.status == BountyProgramRegistry.Status.ACTIVE ||  
4    +    config.status == BountyProgramRegistry.Status.PAUSED ||  
5    +    config.status == BountyProgramRegistry.Status.DRAFT,  
6    "PROGRAM_NOT_ACTIVE"  
7  );
```

Alternatively, add `REJECTED` to `_isBlockingStatus()` while `timeLockEnd` hasn't passed yet.

[H-02] Program deactivation after payout blocks remaining researchers from escalating

HIGH

FIXED

Impact: High**Likelihood:** Medium

Description

The `escalateReport()` function requires the program to be **ACTIVE**:

```
1 // PayoutController.sol:L177-179
2 require(
3     config.status == BountyProgramRegistry.Status.ACTIVE,
4     "PROGRAM_NOT_ACTIVE"
5 );
```

But after `finalizeAndPay()`, the program can transition from **ACTIVE** to **DRAFT** via `deactivateFromPaidBounty()` in `EscrowVault`. This creates a problem when multiple reports exist for the same program.

Consider two active reports on a bounty program. The first gets rejected, the second gets approved and paid out. The payout flows through `finalizeAndPay` -> `payoutBounty` -> `deactivateFromPaidBounty`, moving the program to **DRAFT**. Now the researcher whose report was rejected can no longer escalate - the program isn't **ACTIVE** anymore.

The judge can still call `finalizeAndPay` on the rejected-then-stuck report, but the researcher has no way to challenge the rejection. This creates an asymmetry: judges retain access, researchers lose theirs.

A PoC confirmed this. After Report A's payout (100e6), `bountyBalance` dropped enough to trigger deactivation to **DRAFT**. Researcher 2's `escalateReport` for Report B reverted with `PROGRAM_NOT_ACTIVE`, while the judge could still call `finalizeAndPay` successfully.

Recommendations

Allow escalation from **PAUSED** and **DRAFT** program states for reports still within their timelock window:

```
1 function escalateReport(...) external {
2     BountyProgramRegistry.ProgramConfig memory config = registry.getProgram(programId);
3     require(
4 -         config.status == BountyProgramRegistry.Status.ACTIVE,
5 +         config.status == BountyProgramRegistry.Status.ACTIVE ||
6 +         config.status == BountyProgramRegistry.Status.PAUSED ||
7 +         config.status == BountyProgramRegistry.Status.DRAFT,
8         "PROGRAM_NOT_ACTIVE"
9     );
```

■ Medium Findings

[M-01] No bounty reservation in `finalizeEscalation` creates a race-to-drain

MEDIUM

FIXED

Impact: Medium**Likelihood:** Medium

Description

When `finalizeEscalation()` resolves an escalated report, it checks that enough bounty exists but doesn't actually reserve those funds. So if multiple reports are escalated around the same time, they can each pass the solvency check against the same unreserved pool.

```
1 // PayoutController.sol:L413 - passes for each report, but nothing is set aside
2 require(escrow.bountyBalance(programId) >= payoutAmount, "INSUFFICIENT_BOUNTY");
```

When `finalizeAndPay()` runs later, the first report drains the available bounty. The second report — already resolved by a judge and marked `ESCALATED_RESOLVED` — reverts with `INSUFFICIENT_BOUNTY`. It's now stuck in a blocking status, which means the company can't refund either, not until they deposit more funds.

The result is that payout order determines who gets paid, not who was right.

Recommendations

Reserve the bounty when a report moves to `ESCALATED_RESOLVED`, and release it when payment completes or the report is otherwise closed:

```
1 mapping(uint256 => uint256) public pendingReservations;
2
3 // In finalizeEscalation:
4 pendingReservations[programId] += payoutAmount;
5 require(escrow.bountyBalance(programId) >= pendingReservations[programId], "INSUFFICIENT_BOUNTY")
6     ↪ ;
7
8 // In finalizeAndPay:
9 pendingReservations[programId] -= report.payoutAmount;
```

A simpler alternative: make escalation resolution and payout atomic so the solvency check and fund transfer happen in the same transaction.

[M-02] Second opinion requests are incorrectly blocked when no alternate payout tier exists

MEDIUM

FIXED

Impact: Medium

Likelihood: Medium

Description

The `requestSecondOpinion()` function requires at least one configured severity tier besides the primary judge's severity:

```
1 bool hasAlternateTier = false;
2 for (uint8 s = 1; s <= 4; s++) {
3     if (s == report.primarySeverity) continue;
4     if (registry.payoutBySeverity(programId, s) > 0) {
5         hasAlternateTier = true;
6         break;
7     }
8 }
9 require(hasAlternateTier, "NO_ALTERNATE_PAYOUT_TIER");
```

This is overly restrictive. The second opinion mechanism supports three outcomes: **confirm** (keep the severity), **downgrade** (lower it), and **invalidate** (reject entirely). Only downgrade actually needs an alternate tier. Confirm and invalidate work fine without one.

Real scenario: a program only configures Critical payouts. A judge approves a report as Critical, but the company thinks it's invalid and wants a second opinion to reject it. They can't - no alternate tier exists, even though invalidation doesn't use one.

This effectively strips companies of their dispute rights on single-severity programs.

Recommendations

Remove the `hasAlternateTier` check entirely. The downgrade path in `approveSecondOpinion()` (outcome=1) already handles the case where the chosen severity has no payout by falling back to the lowest configured tier. If no alternate tier exists, the judge simply can't choose outcome=1 — that's enforced naturally by `require(severity < report.primarySeverity, "SEVERITY_NOT_DOWNGRADE")`.

[M-03] Downgrade fallback silently rewrites judge's severity choice

MEDIUM

FIXED

Impact: Medium

Likelihood: Medium

Description

In `approveSecondOpinion()`, the downgrade path doesn't handle unconfigured severities well. If the second judge picks a lower tier that has no configured payout, the contract silently swaps it for the lowest configured tier instead.

```
1 else if (outcome == 1) {
2     payoutAmount = registry.payoutBySeverity(programId, severity);
3     if (payoutAmount == 0) {
```

```
4     uint8 lowest = 0;
5     for (uint8 s = 1; s <= 4; s++) {
6         if (registry.payoutBySeverity(programId, s) > 0) {
7             lowest = s;
8             break;
9         }
10    }
11    severity = lowest;
12 }
13 require(severity < report.primarySeverity, "SEVERITY_NOT_DOWNGRADE");
14 }
```

This leads to two bad outcomes:

1. **Wrong payout:** The researcher gets paid at a severity the judge didn't choose. For example, a program pays High = \$50k and Critical = \$100k. The second judge says "downgrade to Medium," but Medium has no payout, so the code falls back to High and pays \$50k. The judge intended a lower payout, not \$50k.
2. **Permanent deadlock:** In sparse payout schedules where the lowest configured tier equals the primary severity, the fallback produces `severity == primarySeverity`, causing `SEVERITY_NOT_DOWNGRADE` to revert every time. The report gets stuck in `SECOND_OPINION_REQUESTED` indefinitely, a blocking status that prevents the company from refunding.

Recommendations

Don't silently rewrite the judge's chosen severity. If the requested tier has no configured payout, either:

- Revert with a clear error so the judge can pick a valid tier, or
- Treat a downgrade to an unconfigured tier as a zero-payout outcome (the company simply doesn't pay for that severity level)

[M-04] Shared judge pool without per-report reservation blocks settlement

MEDIUMFIXED

Impact: Medium

Likelihood: Medium

Description

The protocol uses a single shared `judgeBalance[programId]` for all reports in a program, but never reserves judge fees per report. This causes two problems:

1. **Immediate invalidation payout:** When `approveSecondOpinion(outcome=2)` invalidates a report, it pays the secondary judge right away from the shared pool. If the balance is already low, the invalidation reverts with `INSUFFICIENT_JUDGE_BALANCE`, leaving the report stuck in `SECOND_OPINION_REQUESTED`.

2. **Delayed payout contention:** Approved reports only pay judges later during `finalizeAndPay()`. If earlier invalidations or payouts drained the pool, finalization reverts with `INSUFFICIENT_JUDGE` and the report gets stuck.

```
1 // Immediate invalidation payout - PayoutController.sol:342-348
2 if (outcome == 2) {
3     uint256 secondaryInvalidationFee = (registry.payoutBySeverity(programId, report.
        ↪ primarySeverity)
4         * config.judgeFeeBps) / 10_000;
5     require(totalJudgeFeeAmount >= secondaryInvalidationFee, "INSUFFICIENT_JUDGE_BALANCE");
6     escrow.payoutJudge(programId, msg.sender, secondaryInvalidationFee);
7 }
```

Each report checks `pool >= myFee` independently, so multiple reports can pass the same check against the same unreserved balance. When one drains the pool, the rest are stuck.

In the worst case, all reports are stuck, the program stays `ACTIVE` (no payout to trigger deactivation), the company can't deposit (requires `DRAFT`), and can't refund (blocking reports exist) — a complete deadlock with no recovery path.

Recommendations

Allow funds to be added directly to the total allocated judge fee for a program.

[M-05] Penalty-only deposit reverts, blocking program reactivation

MEDIUMFIXED

Impact: Medium

Likelihood: Medium

Description

When a program has penalty debt and its `bountyBalance` is below `requiredBounty` (e.g. after a payout drained the pool), the company might try to deposit just enough to clear the penalty and reactivate. But the entire deposit gets consumed by the penalty, leaving `netAmount == 0`. The contract then checks `bountyBalance >= requiredBounty` - which fails and reverts with `INSUFFICIENT_INITIAL_BOUNTY`.

```
1 // EscrowVault.sol:154-161 - penalty eats the deposit first
2 uint256 penaltyPaid = 0;
3 uint256 debt = companyPenaltyDebt[programId];
4 if (debt > 0) {
5     uint256 toPay = amount < debt ? amount : debt;
6     if (toPay > 0) {
7         penaltyPaid = _applyPenaltyPayment(programId, toPay, token, config.treasury);
8     }
9 }
10 uint256 netAmount = amount - penaltyPaid;
```

Since the whole transaction reverts atomically, the penalty isn't actually paid either. The debt persists, the program stays in **DRAFT**, and the company has to figure out on their own that they need to deposit `penalty + (requiredBounty - bountyBalance)` in a single transaction — a constraint that's nowhere in the error message.

The separate `payPenaltyDebt()` function exists but only works when the program is **PAUSED**. For **DRAFT** programs (after payout-triggered deactivation), `deposit()` is the only path, and the penalty-only case is a dead end.

Recommendations

When `netAmount == 0` and `bountyBalance < requiredBounty`, either:

1. Let the penalty payment succeed without trying to activate — split it into a penalty-clearing step and an activation step
2. Revert with a descriptive message like `PENALTY_ONLY_DEPOSIT_INSUFFICIENT_BOUNTY` so the company knows to deposit more
3. Add a `payPenaltyDebt` path for **DRAFT** programs (currently only works for **PAUSED**)

[M-06] Judge fee design biases outcomes toward approval

MEDIUM**FIXED**

Impact: High

Likelihood: Low

Description

The protocol doesn't compensate judges in an outcome-neutral way. In practice, judges earn more when they approve reports and preserve higher severities.

The primary judge only gets a meaningful fee when assigning a non-zero severity in `approvePrimary()`. A full rejection pays nothing — creating a direct incentive to approve borderline reports rather than invalidate them.

The same pattern shows up in second-opinion review. A secondary judge earns more by confirming a higher severity than by downgrading, because the fee is tied to the payout amount of the final severity. Confirming a Critical report pays significantly more than downgrading it to Low. The reviewer is supposed to correct over-rating, but the fee model rewards keeping the higher rating.

Escalation review is asymmetric too. If the second judge rejects an escalation in `finalizeEscalation()`, the code zeros out `judgeFeeAmount`. If confirmed, the judge participates in the normal fee flow. Judges are paid for validating escalations but not for rejecting them.

This isn't an access-control or accounting bug — it's a mechanism-design issue. The protocol relies on judge honesty and off-chain reputation to overcome an on-chain fee structure that consistently favors approval.

Recommendations

Pay judges for review work, not for the outcome they choose.

At minimum:

- Give primary judges a fixed review fee whether they approve or reject
- Base second-opinion compensation on the severity being reviewed, not the downgraded result
- Pay escalation reviewers even when they reject the escalation

[M-07] Active programs can't be topped up with additional deposits

MEDIUM**FIXED**

Impact: Medium

Likelihood: Medium

Description

The first successful `deposit()` activates a bounty program by calling `activateFromDeposit()`, moving it from `DRAFT` to `ACTIVE`. But `deposit()` itself only accepts `DRAFT` programs:

```
1 require(  
2     config.status == BountyProgramRegistry.Status.DRAFT,  
3     "PROGRAM_NOT_DEPOSITABLE"  
4 );
```

Once a program is live, the company can't add more funds to it. They have to wait until the program falls back to `DRAFT` through `deactivateFromPaidBounty()` after the bounty is depleted. There's no proactive top-up path for a running program, which creates operational friction and can make the solvency issues described in M-01 and M-04 worse.

Recommendations

Allow `deposit()` for both `DRAFT` and `ACTIVE` programs, or add a dedicated top-up function for already-active programs.

[M-08] Centralization risk

MEDIUM**FIXED**

Impact: High

Likelihood: Low

Description

The admin role has unchecked powers that can be used to extract funds or block legitimate user operations, with no on-chain guardrails.

Draining bounty funds. The `emergencyBountyPayout()` function lets the admin transfer any amount from any program's bounty balance to any address. There's no restriction on the recipient, no timelock, and no multi-sig requirement at the contract level.

```
1 function emergencyBountyPayout(uint256 programId, address to, uint256 amount) external onlyAdmin
  ↪ {
2   require(to != address(0), "TO_ZERO");
3   require(amount > 0, "AMOUNT_ZERO");
4   uint256 balance = bountyBalance[programId];
5   require(balance >= amount, "INSUFFICIENT_BOUNTY");
6   bountyBalance[programId] = balance - amount;
7   // ... transfers to arbitrary `to` address
8 }
```

The function emits a `BountyPaid` event identical to legitimate payouts, making misuse harder to spot in event logs. It also calls `registry.deactivateFromPaidBounty()`, which moves the program from `ACTIVE` to `DRAFT` — disrupting normal operations as a side effect. The stated purpose is handling stuck payouts, but the implementation provides no guardrails to limit it to that use case.

Blocking company refunds. The `submitReport()` function is admin-only and creates a report in `SUBMITTED` status, which counts as blocking. The `initiateRefund()` function requires zero blocking reports before a company can start a refund. Since the admin controls both the `researcher` and `judge` parameters in `submitReport()`, they can fabricate a report against any `ACTIVE` program at any time — immediately blocking the company from refunding. The fake report will sit in `SUBMITTED` status indefinitely because only the designated judge (controlled by admin) can resolve it.

The flow:

1. Admin calls `submitReport(programId, fakeResearcher, fakeJudge, fakeHash)` — `blockingReportCount` goes up
2. Company calls `initiateRefund(programId)` — reverts with `OPEN_REPORTS_EXIST`
3. The fake report stays in `SUBMITTED` forever since only the admin-chosen judge can approve or reject it

Recommendations

For emergency payouts: add a multi-sig requirement or timelock. Consider giving the program owner a window to approve or veto the call before funds move.

■ Low/Info Findings

[L-01] Fee-on-transfer tokens would break escrow accounting

LOWACKNOWLEDGED

Description

`deposit()` at `EscrowVault.sol:149` credits `bountyBalance` and `judgeBalance` based on the declared `amount` parameter, not the actual tokens received. For fee-on-transfer tokens, the vault receives less than `amount` but records the full value. The gap compounds with every deposit and transfer, making the vault progressively insolvent.

The protocol currently restricts payout tokens via `allowedPayoutToken` (expected to be USDC, which isn't FOT). But the admin can change this to any ERC-20 via `setAllowedPayoutToken()`.

Recommendations

Measure the actual balance change during `transferFrom` in all places:

```
1 + uint256 balanceBefore = token.balanceOf(address(this));
2   require(token.transferFrom(msg.sender, address(this), amount), "TRANSFER_FAILED");
3 + uint256 received = token.balanceOf(address(this)) - balanceBefore;
4 + amount = received;
```

[L-02] No `cancelRefund` function makes `PAUSED` state irreversible

LOW

FIXED

Description

When a company owner calls `initiateRefund()`, the program goes from `ACTIVE` to `PAUSED` and a 5-day refund window starts. The contract even declares a `RefundCancelled` event, confirming that cancellation was planned — but no function ever emits it or restores `ACTIVE` status.

Once a refund is initiated, there's no way to change course. The program stays `PAUSED` for five days no matter what. Companies that accidentally or prematurely trigger `initiateRefund` lose five days of program activity with no recourse.

Recommendations

Add a `cancelRefund` function:

```
1 function cancelRefund(uint256 programId) external {
2   ProgramConfig storage config = programs[programId];
3   require(config.companyOwner == msg.sender, "NOT_PROGRAM_OWNER");
4   require(config.status == Status.PAUSED, "PROGRAM_NOT_PAUSED");
5   require(block.timestamp < config.pausedAt + REFUND_WINDOW, "REFUND_WINDOW_EXPIRED");
6   config.status = Status.ACTIVE;
7   emit RefundCancelled(programId);
8 }
```

[L-03] No resolution deadline for reports after judge assignment

LOW

FIXED

Description

Once a secondary judge is assigned via `assignSecondJudge()`, there's no deadline for them to act. Neither `approveSecondOpinion()` nor `finalizeEscalation()` checks any timestamp. There's no timeout, no admin override to advance a stuck case, and no penalty for delay.

Researchers whose reports are in escalation can't receive their funds until the secondary judge decides to act — regardless of how much time passes.

Recommendations

Store a `judgeDeadline` timestamp in the Report struct when a judge is assigned, and allow admin to force-close stuck escalations after the deadline passes.

[I-01] Never-pruned `programPenaltyReports` causes unbounded gas growth

INFORMATIONAL

ACKNOWLEDGED

Description

`programPenaltyReports[programId]` grows every time `accrueCompanyPenalty()` is called, but entries are never removed after their penalty debt is fully paid. `_applyPenaltyPayment()` still iterates over the full historical array on every settlement, even when most entries are already zeroed out.

This affects all penalty-settlement paths: `deposit()`, `payPenaltyDebt()`, and `refund()`. Over time, these operations get progressively more expensive. On long-lived programs with many penalized reports, settlement could eventually become impractical within realistic gas limits.

```
1  programPenaltyReports[programId].push(reportId);
2
3  bytes32[] storage reportIds = programPenaltyReports[programId];
4  uint256 reportCount = reportIds.length;
5  for (uint256 i = 0; i < reportCount; i++) {
6      bytes32 rId = reportIds[i];
7      uint256 reportDebt = reportPenaltyDebt[rId];
8      if (reportDebt == 0) continue;
9      // ...
10 }
```

Recommendations

Prune fully-paid entries from `programPenaltyReports[programId]` using a swap-and-pop or lazy-compaction approach, so settlement cost scales with active debts rather than historical ones.

[I-02] Faucet `Withdrawn` event logs wrong amount

INFORMATIONAL

ACKNOWLEDGED

Description

In `src/Faucet.sol`, the admin `withdraw()` function sends the full contract balance to `msg.sender`, then emits `Withdrawn(msg.sender, address(this).balance)`. Since the balance is read after the transfer, the event always records 0 instead of the actual withdrawn amount.

No funds are lost. The ETH transfers correctly, but indexers and off-chain tools relying on this event will see an incorrect amount.

Recommendations

Capture the balance before the transfer and emit that value:

```
1 uint256 amount = address(this).balance;
2 (bool ok,) = payable(msg.sender).call{value: amount}("");
3 require(ok, "ETH_TRANSFER_FAILED");
4 emit Withdrawn(msg.sender, amount);
```